

Save History with Git Commit

Git Repository Basics

Learning objective

Create a clear snapshot of staged changes with a meaningful commit message.

Core idea

A commit records the staged changes as a named snapshot in the repository history. Good commits are focused, understandable, and easy to review or reverse later.

Commands to know

Command / pattern	Purpose
<code>git commit -m "Add lesson page"</code>	Commit staged changes
<code>git log --oneline</code>	View compact commit history
<code>git show</code>	Inspect the latest commit

Step-by-step workflow

- 1. Use **git status** to review your work.
- 2. Stage related changes with **git add**.
- 3. Commit them with a short message describing what changed.
- 4. Use **git log --oneline** to confirm the commit.

Common mistakes

- Writing unclear messages such as **update** or **fix**.
- Combining unrelated changes in one commit.
- Expecting unstaged changes to be included automatically.

Practice task

Create a small file, stage it, and commit it with the message **Add practice notes**. Confirm it using **git log --oneline**.

Quick check

Which changes are included when you run `git commit`: all files or only staged changes?

Tip: Run `git status` before and after important Git commands so you can observe what changed.